Retail Orders: Queries by Hand

Fall 26

Calvin Williamson, Science and Math

The retail orders database has three tables. Each order is one customer buying one product. `customer_id` and `product_id` in `orders` point back to the `customers` and `products` tables.

Work out what each query returns **by hand**, using only the tables below. Write the result as a small table. Then check your answers by running the queries on the database.

The tables

customers

customer_id	first_name	last_name
1	John	Doe
2	Jane	Smith
3	Robert	Johnson
4	Sarah	Williams

products

product_id	name	price
1	Silk Blouse	89.99
2	Denim Jeans	79.99
3	Cashmere Sweater	149.99
4	Cotton T-Shirt	29.99
5	Wool Coat	199.99

orders

order_id	customer_id	product_id	order_date
1	1	3	2025-03-10
2	2	1	2025-03-12
3	3	4	2025-03-15
4	2	4	2025-03-18
5	1	2	2025-03-20
6	2	3	2025-03-25

Level 1: one table

1. Products under $100.

```sql
SELECT name, price FROM products WHERE price < 100;
```

2. Products from most to least expensive.

```sql
SELECT name, price FROM products ORDER BY price DESC;
```

3. Jane's orders. (Find her `customer_id` first.)

```sql
SELECT order_id, order_date FROM orders WHERE customer_id = 2;
```

4. Orders on or after March 15.

```sql
SELECT COUNT(*) FROM orders WHERE order_date >= '2025-03-15';
```

5. The average, highest, and lowest price.

```sql
SELECT AVG(price), MAX(price), MIN(price) FROM products;
```

Level 2: joins

6. Each order with the customer's name and the product name.

```
SELECT o.order_id, c.first_name, p.name
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN products p  ON p.product_id  = o.product_id
ORDER BY o.order_id;
```

7. Who bought the Cashmere Sweater?

```
SELECT c.first_name, c.last_name
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN products p  ON p.product_id  = o.product_id
WHERE p.name = 'Cashmere Sweater';
```

8. Total sales across all orders.

```
SELECT SUM(p.price)
FROM orders o JOIN products p ON p.product_id = o.product_id;
```

Level 3: GROUP BY

9. How much each customer spent.

```sql
SELECT c.first_name, SUM(p.price) AS total
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN products p  ON p.product_id  = o.product_id
GROUP BY c.first_name;
```

10. Number of orders per customer, including customers with none.

```sql
SELECT c.first_name, COUNT(o.order_id) AS num_orders
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.first_name;
```

11. How many times each product was ordered.

```sql
SELECT p.name, COUNT(o.order_id) AS times_ordered
FROM products p
LEFT JOIN orders o ON o.product_id = p.product_id
GROUP BY p.name;
```

Think about it

12. Run query 10 again with `JOIN` instead of `LEFT JOIN`. What changes, and why?

13. How many rows would this query return? Explain how you got your answer.

```
SELECT * FROM customers CROSS JOIN products;
```

14. Query 10 uses `COUNT(o.order_id)`. What would Sarah's count be if it used `COUNT(*)` instead, and why?